AI Engineering con Python

El objeto de este tema es programar en Python un Agente IA.

Agente IA: **es un sistema autónomo** que planifica, razona y utiliza herramientas (incluido RAG) para completar tareas.

Pasos básicos

Los pasos básicos para la creación de un agente IA son:

1. **Definir el propósito**: Determinar las tareas específicas que realizará el agente, como investigar en la web, analizar archivos PDF o automatizar el navegador.
2. **Configurar el entorno**: Instalar Python y las librerías necesarias con pip. Se necesita una clave de API (OpenAI, Google Gemini, o Anthropic) para el "cerebro" del agente.
3. **Seleccionar el LLM**: Elegir el modelo de lenguaje que procesará las instrucciones.
4. **Definir herramientas**: Crear funciones en Python que el agente pueda invocar, como una función para buscar en Google o realizar cálculos matemáticos.
5. **Diseñar el flujo de trabajo**: Definir cómo el agente recibirá la tarea, pensará en el siguiente paso, usará la herramienta y evaluará el resultado hasta que termine.

Más info sobre agentes IA

En este documento se estructura en 4 puntos,

1. Fundamentos y APIs de LLMs
2. Embeddings y Bases de Datos Vectoriales
3. El Flujo RAG (Retrieval Augmented Generation)
4. El Agente Inteligente con Memoria y Herramientas

Cada uno de ellos tiene:

- una introducción teórica
- un ejercicio resuelto y explicado
- ejercicio propuesto para entregar

Comenzamos:

1.Fundamentos y APIs de LLMs

- **Teoría:** ¿Qué es un LLM? Tokens, Temperature, Context Window y System Prompts.
- **Práctica:** Uso de la librería Google en Python.
- **Reto:** Crear un script que toma un texto largo y lo resume en 3 puntos clave con un tono específico (ej. "estilo pirata").

---

1. LLM (Large Language Model)

Es un modelo de Inteligencia Artificial entrenado con volúmenes masivos de texto para comprender, generar y manipular el lenguaje humano. No "piensa", sino que predice la siguiente palabra (token) más probable en una secuencia basándose en patrones estadísticos complejos.

Los modelos no consultan una base de datos interna para responder, sino que 'recuerdan' relaciones entre palabras aprendidas durante su entrenamiento (un proceso que consume petabytes de datos y meses de computación). Esto ayuda a entender por qué a veces "alucinan".

- **Ejemplos:** GPT-4 (OpenAI), Llama 3 (Meta), Claude (Anthropic) o Gemini (Google).

2. Token

Es la unidad básica de procesamiento de un LLM. No siempre es una palabra completa; puede ser una sílaba, un carácter o un signo de puntuación. Los modelos tienen un **límite de tokens** (ventana de contexto) que determina cuánto texto pueden "leer" y "escribir" de una sola vez.

3. API (Application Programming Interface)

Es el "puente" que permite que tu programa de Python se comunique con el cerebro de la IA que reside en un servidor remoto. A través de la API, enviamos una solicitud (**prompt**) y recibimos una respuesta estructurada.

4. Temperatura

Es un parámetro de configuración que controla la "creatividad" o aleatoriedad del modelo:

- Usa **Temperatura 0** para generar JSON, SQL o resumir leyes.

- Usa **Temperatura 0.7 - 1.0** para brainstorming de nombres de variables o redactar correos creativos

5. Prompt Engineering

Es el arte de diseñar instrucciones precisas para obtener el mejor resultado del modelo. Un buen prompt suele incluir: **Rol** (Eres un experto...), **Contexto** (Basándote en este PDF...), **Tarea** (Extrae la duración...) y **Formato** (Responde en una tabla...).

6. Inferencia

Es el proceso en el que el modelo ya entrenado recibe un dato de entrada y genera una salida. Cuando ejecutas tu código y Gemini te responde, estás realizando una **inferencia**.

Una **IA local** es un modelo de inteligencia artificial que se ejecuta directamente en tu propio hardware (ordenador, servidor, móvil) en lugar de depender de servidores en la nube como ChatGPT. Esto garantiza privacidad total, ya que tus datos no salen de tu dispositivo, no requiere conexión a internet y evita costes de suscripción.

Requisitos y Herramientas: Ejecutar IA localmente exige potencia gráfica (GPU) o de procesador (CPU). Herramientas como Ollama, LM Studio o AnythingLLM facilitan la descarga y uso de modelos abiertos (Llama 3, Mistral, Gemma) en tu equipo.

Comparativa: IA en la Nube vs. IA Local

Característica	API en la Nube (Groq / DeepSeek)	IA Local (Ollama / Llama.cpp)
Velocidad	**Ultra rápida:** Usa hardware especializado (LPU/GPU externas).	**Variable:** Depende totalmente de la potencia de tu procesador y RAM.
Coste	Generalmente de pago por uso	**Gratis:** Una vez descargado el modelo, no pagas por cada pregunta.

Privacidad	Los datos viajan por internet a servidores externos.	**Máxima:** Los datos nunca salen de tu ordenador.
Instalación	Solo necesitas una API Key y conexión a internet.	Requiere descargar archivos grandes (GB) y configurar el entorno.
Dependencia	Si no tienes internet o el servidor cae, no hay IA.	Funciona 100% offline (en un avión, en el campo, etc.).

El Resumidor Pirata (Fundamentos de LLM y LCEL)

Enlace a los ejercicios:
https://github.com/salvadormartos/IAGenerativa.git

Una vez descargado el código tienes que:

- En el directorio principal del proyecto, crear una archivo .env con GOOGLE_API_KEY=tu_api_key. Con tu cuenta personal de google y accediendo a aistudio.google.com puedes obtener una API Key.
- Instalar paquetes:
  - pip install -r requirements.txt
  - pip install pypdf
  - pip install sentence-transformers
  - pip install faiss-cpu

ejercicio1.py

```
import os
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from dotenv import load_dotenv

# 1. Preparar las credenciales
load_dotenv()
# Asegúrate de que en tu .env la variable se llame GOOGLE_API_KEY
api_key = os.getenv("GOOGLE_API_KEY")

def resumidor_pirata():
    # 2. Inicializar el LLM
    llm = ChatGoogleGenerativeAI(
```

```
        model="gemini-2.5-flash-lite",
        temperature=0,
        max_tokens=None,
        timeout=None,
        max_retries=2,
    )

    # 3. Definir la "Personalidad" y la "Tarea" (Prompt Engineering)
    system_msg = "Eres un pirata de los siete mares experto en leyes educativas. Tu misión
es resumir textos aburridos en 3 puntos clave usando jerga pirata."

    prompt = ChatPromptTemplate.from_messages([
        ("system", system_msg),
        ("human", "Resume este texto en 3 puntos clave: {texto_a_resumir}")
    ])

    # 4. Crear la cadena simple (Chain)
    chain = prompt | llm

    # 5. El texto de prueba
    texto_ejemplo = """
    El módulo de Proyecto de desarrollo de aplicaciones Web tiene una duración total
    de 40 horas y se imparte en el segundo curso del ciclo. Su evaluación se
    realizará una vez cursados el resto de módulos profesionales y requiere la
    elaboración de un producto final que integre las competencias adquiridas.
    """

    # 6. Ejecutar
    print("--- Procesando botín de información con Gemini... ---\n")
    respuesta = chain.invoke({"texto_a_resumir": texto_ejemplo})

    print(respuesta.content)

if __name__ == "__main__":
    resumidor_pirata()
```

Este script introduce el concepto de **Prompt Engineering** y el uso de **Cadenas (Chains)** en LangChain. Es la base para entender cómo una IA puede adoptar un rol específico y procesar información estructurada.

Explicación Detallada

Componente	Función
load_dotenv()	**Seguridad primero.** Carga la API Key desde un archivo .env. Nunca debemos escribir las llaves

	directamente en el código para evitar "fugas" en repositorios públicos. **Importante:** Asegúrate de crear un archivo `.gitignore` e incluir la línea `.env` dentro. Si subes tu API Key a GitHub, los bots la robarán en segundos y agotarán tu cuota.
model="gemini-2.5-flash-lite"	El "cerebro". Usamos la versión *lite* por su velocidad y eficiencia en tareas de transformación de texto sencillas.
temperature=0	**Determinismo.** En 0, la IA es "seria" y precisa. En 1, es creativa y aleatoria. Para resumir datos reales (horas, módulos), siempre usamos 0.
ChatPromptTemplate	Define la estructura de la conversación. Separamos el **System Message** (instrucciones de comportamiento) del **Human Message** (los datos que el usuario aporta).

Ejercicio Propuesto: "El Traductor de Jerga Técnica"

Objetivo: Crear un script que tome un error de código complejo y lo explique a diferentes públicos usando variables.

Instrucciones para el alumno:

1. Modifica el `ChatPromptTemplate` para que acepte **dos variables**: `{error_tecnico}` y `{nivel_del_publico}` (por ejemplo: "niño de 5 años", "CEO con prisa" o "estudiante de DAW").
2. El `system_msg` debe obligar a la IA a actuar como un **traductor técnico**.
3. El resultado debe ser un mensaje que no solo explique el error, sino que proponga una **solución en una sola línea de código**.

¿Cómo sabrás que está bien? Si al pasarle un "*NullPointerException*" y decirle que el público es un "niño", la IA usa una analogía (como un juguete perdido o una caja vacía).

Pista técnica: "Recuerda que para pasar dos variables, tu `chain.invoke` ahora debe ser: `chain.invoke({"error_tecnico": var1, "nivel_del_publico": var2})`".

2.Embeddings y Bases de Datos Vectoriales

- **Teoría:** Cómo "entiende" la IA los conceptos (vectores). Introducción a **FAISS, ChromaDB** o **Pinecone.**
- **Práctica:** Convertir una lista de frases en números (embeddings) y buscar las más parecidas semánticamente.
- **Reto:** Un buscador "inteligente" de una biblioteca de libros (buscas por "magia y huérfanos" y te sale Harry Potter).

Entramos en el verdadero "corazón" de la ingeniería de IA moderna. Aquí es donde pasamos de simplemente "chatear" con un modelo a construir sistemas que poseen memoria técnica.

1. ¿Qué es un Embedding?

Imagina que quieres que un ordenador entienda la diferencia entre "Perro" y "Gato". Para una máquina, son solo letras. Un **Embedding** es la traducción de una palabra o frase a una **lista de números** (un vector).

Lo fascinante es que estos números no son aleatorios:

- Palabras con significados similares terminan con números similares.
- En un espacio matemático, la distancia entre el vector "Rey" y "Hombre" es la misma que entre "Reina" y "Mujer".

Definición corta: Un embedding es una representación numérica del **significado semántico** de un texto.

2. Bases de Datos Vectoriales (Vector Stores)

Las bases de datos tradicionales (como SQL o Excel) buscan coincidencias exactas de palabras. Las **Bases de Datos Vectoriales** buscan "significados".

Su función en un sistema RAG es:

1. **Almacenar** los vectores (embeddings) de miles de trozos de tus documentos.
2. **Comparar** el vector de la pregunta del usuario con todos los guardados.
3. **Recuperar** los fragmentos que están matemáticamente más "cerca" de la intención del usuario.

3. Herramientas Principales: FAISS, ChromaDB y Pinecone

Existen diferentes "sabores" de estas bases de datos según la necesidad:

Herramienta	Tipo	Uso ideal para el curso
FAISS	Local (En memoria)	**La que usamos en el ejemplo.** Es ligera, no requiere registro y es perfecta para prototipos y archivos individuales.
ChromaDB	Local / Open Source	Ideal si queremos que la base de datos se guarde en una carpeta del PC de forma permanente y sea fácil de consultar después.
Pinecone	Cloud	Se usa en empresas. Los vectores viven en la nube. Es escalable pero requiere conexión y gestión de API Keys.

Conceptos Técnicos que debes de entender:

- **Chunking (Fragmentación):** El proceso de dividir un PDF largo en trozos pequeños para que el embedding sea preciso. Si el **chunk** es muy pequeño, el vector no tiene suficiente información para saber el significado. Si es muy grande, el vector se vuelve 'genérico' y pierde precisión. El equilibrio es la clave.
- **Cosine Similarity (Similitud del Coseno):** La fórmula matemática más común para medir qué tan cerca están dos vectores entre sí.
- **Top-K:** El número de fragmentos (ej. k=5) que el sistema decide "traer" de la base de datos para responder a una pregunta.

Ejercicio Resuelto

Este script hace lo siguiente:

1. Tome 5 palabras muy distintas, genera sus vectores y muestra cómo la frase de "fútbol" tiene una puntuación de similitud mucho mayor con "gol" que con "planeta".
2. Dado una serie de libros con su temática, localiza uno de ellos.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS


def biblioteca_inteligente_con_metricas():
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

    # 1. Creamos una base de datos rápida con palabras para comparar
    # La diferencia entre "planeta" y "Planeta"
    # demuestra que el modelo es case senstitive
    # aunque mantenga la cercanía semántica.

    palabras_test = ["gol", "planeta", "grada", "libro", "Planeta"]
    vector_db = FAISS.from_texts(palabras_test, embeddings)

    print("--- COMPARACIÓN DE DISTANCIAS SEMÁNTICAS ---")
    query_palabra = "futbol"

    # Buscamos las distancias de "futbol" contra nuestra base
    # k=4 para ver todas las palabras del test
    resultados = vector_db.similarity_search_with_score(query_palabra, k=5)

    print(f"Palabra de origen: '{query_palabra}'\n")
    print(f"{'Palabra destino':<15} | {'Distancia (Menor es mejor)':<25}")
    print("-" * 45)

    for doc, score in resultados:
        # El 'score' es la distancia Euclidiana
        print(f"{doc.page_content:<15} | {score:.4f}")

    print("\n" + "=" * 50)
    print("--- BUSCADOR DE LIBROS ---")
    libros = [
        "Harry Potter: Un niño huérfano descubre que es mago y asiste a una escuela de hechicería.",
        "El Señor de los Anillos: Un hobbit debe destruir un anillo de poder en un mundo de fantasía
épica.",
        "Steve Jobs: Biografía del fundador de Apple y su revolución en la tecnología personal."
    ]
    vector_db_libros = FAISS.from_texts(libros, embeddings)

    query_libro = "un chico sin padres que hace trucos de magia"
    # Recuperamos el doc y su score
```

```
    doc_libro, score_libro = vector_db_libros.similarity_search_with_score(query_libro, k=1)[0]

    print(f"Buscando: '{query_libro}'")
    print(f"Recomendación: {doc_libro.page_content}")
    print(f"Confianza (Distancia): {score_libro:.4f}")

if __name__ == "__main__":
    biblioteca_inteligente_con_metricas()
```

Este es el script fundamental para entender el "corazón" de un sistema RAG. Aquí dejamos de trabajar con palabras y empezamos a trabajar con **conceptos matemáticos**. Es la base de cómo una IA "entiende" que un hobbit tiene más que ver con un mago que con un iPhone.

Este script introduce la **búsqueda semántica**. A diferencia de un buscador tradicional (Ctrl+F) que busca letras exactas, este sistema busca **significados** transformando el texto en vectores (listas de números).

Explicación Detallada

Componente	Función
HuggingFaceEmbeddings	El traductor. Convierte texto humano en un **Vector** (una coordenada en un espacio de cientos de dimensiones). Usamos el modelo all-MiniLM-L6-v2 por ser ligero y excelente para textos cortos y se ejecuta en la RAM de tu ordenador. Es instantáneo y gratuito para procesar miles de frases sin depender de la conexión a internet ni de créditos de API
FAISS	La biblioteca de búsqueda. Es una base de datos diseñada por Facebook para encontrar rápidamente qué vectores están "cerca" de otros en ese espacio multidimensional.
similarity_search_with_score	Además de buscar, nos devuelve el **Score** (la Distancia Euclidiana).

Distancia (Score)	**Concepto Clave:** En FAISS, **menor es mejor.** Un score de 0.0 significaría que los textos son idénticos. Cuanto más bajo sea el número, más se parecen semánticamente.

Que debemos notar:

Es crucial que se observe la tabla de distancias de "fútbol":

- Fíjate que **"gol"** y **"grada"** tienen distancias menores que **"planeta"**. La IA sabe que pertenecen al mismo contexto deportivo.
- Nota la diferencia entre **"planeta"** (minúscula) y **"Planeta"** (mayúscula). Aunque el modelo suele ser robusto, el *case sensitive* puede influir ligeramente en la posición del vector.

Lección: La búsqueda semántica permite encontrar "Harry Potter" aunque la pregunta sea "chico que hace trucos", sin que coincida ni una sola palabra. Esto es lo que permite que un RAG responda preguntas aunque el usuario no use los términos técnicos del PDF.

Glosario

- **Distancia Euclidiana:** La longitud del segmento que une dos puntos. Es el "score" que nos da FAISS.
- **Semántica:** El estudio del significado. La IA vectorial es "Semántica" porque no busca letras, busca conceptos.

Ejercicio Propuesto: "El Filtro de Candidatos (HR-Tech)"

Objetivo: Crear un sistema que clasifique candidatos para una oferta de trabajo basándose en su descripción, no en palabras clave.

Instrucciones para el alumno:

1. Crea una lista de 5 "Perfiles de Candidatos" (ej: "Experto en Python y backend con 5 años de experiencia", "Diseñador gráfico especializado en interfaces móviles", etc.).
2. Crea una base de datos vectorial con esos perfiles.

3. Define una query_vacante que sea: "Buscamos un programador para crear servidores y lógica de negocio".
4. El script debe devolver los **3 mejores candidatos** ordenados por su score de confianza.
5. **Bonus:** Añade un "Candidato Trampa" que use la palabra "servidor" pero en otro contexto (ej: "Camarero con experiencia sirviendo mesas") y comprueba si la IA es capaz de ponerlo al final de la lista a pesar de la coincidencia de palabras.

3.El Flujo RAG (Retrieval Augmented Generation)

1. ¿Qué es RAG?

RAG (Retrieval-Augmented Generation o Generación Aumentada por Recuperación) **es una técnica** de IA que mejora las respuestas de los modelos de lenguaje (LLM) al permitirles consultar fuentes de datos externas y actualizadas —como documentos privados o internet— antes de generar una respuesta, reduciendo así las alucinaciones y aumentando la precisión.

Puntos clave sobre RAG:

¿Cómo funciona? Cuando haces una pregunta, el sistema busca información relevante en una base de datos propia (documentos, PDFs, bases de datos), añade esta información como "contexto" a tu pregunta y se la pasa al modelo de IA (como GPT) para que genere una respuesta informada.

Ventajas:

- **Información actualizada**: No depende solo de los datos de entrenamiento del modelo.
- **Menos alucinaciones**: Al basarse en documentos reales, las respuestas son más precisas.
- **Seguridad**: Permite usar datos privados o corporativos de forma segura.
- **Cita fuentes**: El modelo puede indicar de qué documento obtuvo la información.
- **¿Para qué se usa?** Es ideal para chatbots de atención al cliente, asistentes virtuales que acceden a manuales técnicos, búsquedas en

documentos legales o cualquier tarea que requiera conocimientos específicos y actuales.

En resumen, RAG combina la capacidad creativa de un modelo de IA con la precisión de una búsqueda de información específica, haciéndolo más "culto" y fiable sin tener que reentrenarlo, que es caro y lento.

2. Las 3 Fases del Flujo RAG

Para entenderlo, podemos dividir el proceso en tres grandes bloques:

A. Fase de Ingesta (Preparar los datos)

Es lo que ocurre antes de hacer preguntas. Solo se hace una vez por documento.

1. **Carga:** Leemos el archivo (PDF, TXT, Web).
2. **Chunking:** Dividimos el texto en trozos (chunks).
3. **Embedding:** Convertimos cada trozo en un vector numérico.
4. **Almacenamiento:** Guardamos esos vectores en el Vector Store (FAISS). Debemos aclarar que, en nuestro script, la base de datos FAISS vive en la **memoria RAM**. Si cierras el programa, los vectores desaparecen y hay que volver a indexar el PDF. En aplicaciones reales, esto se guarda en disco.

B. Fase de Recuperación (Retrieval)

Es lo que ocurre cuando el usuario hace una pregunta.

1. La pregunta se convierte en un vector (usando el mismo modelo de embeddings).
2. El sistema busca en la base de datos los **Top-K** trozos de texto que más se parecen a la pregunta.

C. Fase de Generación (Augmentation)

Es el paso final donde entra el "cerebro".

1. Se construye un **Super-Prompt** que incluye: *"Aquí tienes esta información extraída del PDF [Contexto]. Basándote en ella, responde a la pregunta [Pregunta]"*.
2. El LLM genera la respuesta final.

---

3. Conceptos:

- **Hallucination (Alucinación):** Ocurre cuando la IA inventa un dato. El RAG reduce drásticamente las alucinaciones porque obligamos a la IA a citar el contexto.
- **Context Window (Ventana de Contexto):** Es el espacio total (en tokens) que la IA puede procesar. Si el contexto recuperado es muy grande, "se sale" por los bordes y el modelo da error o ignora información.
- **Prompt Template:** Es el molde predefinido que usamos para inyectar el contexto de forma limpia.

Ejercicio Resuelto:

ejercicio3.py

```
import os
from dotenv import load_dotenv
from langchain_classic.chains.combine_documents import create_stuff_documents_chain
from langchain_classic.chains.retrieval import create_retrieval_chain
from langchain_google_genai import ChatGoogleGenerativeAI

from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import FAISS
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.prompts import ChatPromptTemplate

load_dotenv()
# Asegúrate de que en tu .env la variable se llame GOOGLE_API_KEY
api_key = os.getenv("GOOGLE_API_KEY")

def run_rag_debug():
    # 1. Configuración

    # 2. Inicializar el LLM
    llm = ChatGoogleGenerativeAI(
        model="gemini-2.5-flash-lite",  # O "gemini-2.5-pro" si quieres más potencia
        temperature=0,
        max_tokens=None,
        timeout=None,
        max_retries=2,
    )

    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

    # 2. Carga
    archivo = "Rivas-Guia_basica_uso_inteligencia_artificial_generativa_2025.pdf"
    if not os.path.exists(archivo):
        print(f"Error: No se encuentra {archivo}")
        return

    loader = PyPDFLoader(archivo)
```

```
    docs = loader.load()
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
    chunks = text_splitter.split_documents(docs)
    vector_store = FAISS.from_documents(chunks, embeddings)

    # 3. Diseño del Prompt (Versión para Gemini)
    prompt = ChatPromptTemplate.from_template("""
        Utiliza el siguiente contexto para responder a la pregunta del usuario.
        Si la respuesta no está literal, intenta deducirla basándote en la información disponible.
        Si el contexto no tiene absolutamente nada que ver, indica qué temas se tratan en el
texto.

        Contexto: {context}
        Pregunta: {input}
        """)

    # 4. Creación de la Cadena
    combine_docs_chain = create_stuff_documents_chain(llm, prompt)
    # El retriever actúa como el 'filtro' que decide qué 5 trozos (k=5 por defecto)
    # de los miles que hay en el PDF son los que Gemini debe leer.
    rag_chain = create_retrieval_chain(vector_store.as_retriever(), combine_docs_chain)

    # 5. Ejecución con Auditoría
    pregunta = " Un hospital implementa una IA para ayudar a los radiólogos. El KPI Reducción de
la tasa de error diagnóstico en un 5%, es un ejemplo de:"

    print(f"\n--- Preguntando: {pregunta} ---")

    response = rag_chain.invoke({"input": pregunta})

    # --- IMPRIMIR EL CONTEXTO ---
    print("\n" + "=" * 50)
    print("CONTEXTO RECUPERADO (Lo que el buscador encontró)")
    print("=" * 50)

    # Recorremos los documentos recuperados
    for i, doc in enumerate(response["context"]):
        print(f"\nFRAGMENTO {i + 1} (Página {doc.metadata.get('page', 'N/A')}):")
        print("-" * 30)
        print(doc.page_content)  # Aquí imprimimos el texto real del trozo
        print("-" * 30)

    print("\n" + "=" * 50)
    print("RESPUESTA FINAL DE LA IA")
    print("=" * 50)
    print(response["answer"])


if __name__ == "__main__":
    run_rag_debug()
```

Este es el "corazón" de la arquitectura RAG. Aquí es donde unimos el cerebro (LLM) con los datos (PDF). Es el script más importante para

entender que **una IA solo es tan buena como la información que el "buscador" le pone sobre la mesa.**

Este script enseña el flujo completo de una aplicación de IA con datos externos: **Cargar -> Trocear -> Indexar -> Recuperar -> Responder.**

Explicación Detallada

Componente	Función
PyPDFLoader	Extrae el texto plano del archivo PDF y mantiene los metadatos (como el número de página).
RecursiveCharacterTextSplitter	**La técnica de la "Criba".** Corta el texto en fragmentos (*chunks*). El chunk_overlap es vital para que un concepto no se corte a la mitad y pierda el sentido entre un trozo y otro.
create_stuff_documents_chain	Define cómo se "empaquetan" los documentos recuperados dentro del prompt para que el LLM los lea.
create_retrieval_chain	El coordinador. Recibe la pregunta, llama al buscador (Retriever), le pasa los fragmentos al LLM y nos da la respuesta.

Explicación con más detalles de los puntos 4 y 5 del código, que es donde ocurre la “magia” de la integración:

1. combine_docs_chain (tipo Chain (Runnable))
La función: create_stuff_documents_chain(llm, prompt)

- **¿Qué es?**: Es la cadena encargada de **generar la respuesta.**
- **¿Qué contiene?**: Un objeto "Runnable" que sabe hacer una sola cosa: recibir una lista de documentos (fragmentos de tu PDF) y "embutirlos" (de ahí el nombre *stuff*) dentro del {context} del prompt para enviárselos al LLM. Es el método más rápido, pero tiene el límite de la *Context Window* del modelo

2. rag_chain (tipo Chain (Master))
La función: create_retrieval_chain(vector_store.as_retriever(), combine_docs_chain)

- **¿Qué es?**: Es la **cadena maestra** (End-to-End).
- **¿Qué contiene?**: Un flujo de trabajo completo.
    1. Primero usa el retriever para buscar en la base de datos vectorial (vector_store).
    2. Luego pasa esos documentos encontrados a la combine_docs_chain que creamos antes.

3. pregunta (tipo String)
- **¿Qué contiene?**: Simplemente un **String** (cadena de texto).

4. response (tipo Dictionary)
La función: rag_chain.invoke({"input": pregunta})

Aquí es donde se ejecuta todo. Al llamar a .invoke(), la variable response se convierte en un **Diccionario de Python** con una estructura muy específica.

¿Qué contiene exactamente response?:

- **response["input"]**: Contiene el texto original de tu pregunta.
- **response["context"]**: Esta es la joya de la corona. Es una **Lista de objetos de clase Document**. Contiene los 4 o 5 fragmentos exactos del PDF que el buscador consideró relevantes. Cada uno tiene su page_content (el texto) y sus metadata (número de página).
- **response["answer"]**: Contiene un **String** con la respuesta final que redactó Gemini tras leer el contexto.

Buenas Prácticas de RAG:

1. **Calidad del PDF:** Si el PDF es una imagen (escaneado), el PyPDFLoader fallará. Se necesita texto real.
2. **Chunk Overlap:** El solapamiento es como el "solape" de las tejas de un tejado; evita que la información importante se escape por las grietas entre fragmentos.

Ejercicio Propuesto: "El Detective de Chunks"

Objetivo: Comprender cómo el tamaño de los fragmentos afecta a la capacidad de respuesta de la IA.

Instrucciones para el alumno:

1. Utiliza el mismo PDF pero cambia los valores de la configuración de troceado:
    - **Prueba A:** `chunk_size=100`, `chunk_overlap=0` (Trozos minúsculos).
    - **Prueba B:** `chunk_size=5000`, `chunk_overlap=500` (Trozos enormes).
2. Realiza la misma pregunta técnica: *"¿Qué es un KPI y pon un ejemplo de la guía?"*.
3. **Analiza:**
    - En la **Prueba A**, ¿la respuesta es coherente o está "rota" porque la información está demasiado repartida?
    - En la **Prueba B**, ¿tarda más en responder? ¿El contexto recuperado incluye demasiada paja (información irrelevante)?
4. **Reto Extra:** Modifica el prompt para que, además de la respuesta, la IA te diga siempre en qué página del PDF encontró la información (usando los metadatos del contexto).

Pista: Para acceder a la página, investiga el atributo `doc.metadata['page']` dentro del bucle de respuesta. (Nota: Recordar que las páginas en los metadatos suelen empezar en 0).

4.El Agente Inteligente con Memoria y Herramientas

1. ¿Qué es un Agente (AI Agent)?

Hasta ahora, nuestro sistema RAG es pasivo: tú preguntas y él busca en el PDF. Un **Agente** es un LLM que tiene acceso a "herramientas" y decide por sí mismo cuál usar.

- **Sin Agente:** El programa siempre busca en el PDF.
- **Con Agente:** Si preguntas algo del PDF, usa la herramienta de búsqueda. Si preguntas "¿Qué hora es en Tokio?", decide usar una herramienta de Google o de reloj. Si pides un cálculo, usa una calculadora.

> **Definición corta:** Un agente es un LLM con capacidad de razonamiento que utiliza herramientas externas para completar una tarea compleja.

2. Herramientas (Tools)

Las herramientas son funciones de Python que el LLM puede invocar. Esto es muy potente porque pueden conectar la IA con el mundo real:

- **Search Tool:** Buscar en Wikipedia o Google.
- **Python Tool:** Escribir y ejecutar código para resolver un problema matemático.
- **Custom Tool:** Una función que tú escribas (por ejemplo, consultar la base de datos de alumnos del instituto).

3. Evaluación y Mejora (RAG Evaluation)

¿Cómo sabemos si nuestro RAG es bueno? Usamos métricas:

- **Fidelidad (Faithfulness):** ¿La respuesta se basa *realmente* en el PDF o la IA se lo ha inventado?
- **Relevancia:** ¿El fragmento recuperado sirve para responder a la pregunta?
- **Precisión de la respuesta:** ¿Es el dato exacto?

4. Resumen del Pipeline Profesional (Mantenimiento)

Para que un ingeniero de IA entregue un proyecto, debe cumplir este ciclo:

1. **Logging:** Guardar todas las preguntas y respuestas para detectar fallos.
2. **Feedback Loop:** Permitir que el usuario marque si la respuesta fue útil.
3. **Iteración:** Ajustar el chunk_size o el prompt basándose en los errores detectados.

ejercicio4.py

```
import os
import sys
from dotenv import load_dotenv
from langchain_classic.agents import create_tool_calling_agent, AgentExecutor

# --- IMPORTACIONES ---
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.tools import create_retriever_tool
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory

from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_community.vectorstores import FAISS
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

load_dotenv()

def configurar_asistente():
    if not os.path.exists("normativa"):
        os.makedirs("normativa")
```

```
        print("Crea la carpeta 'normativa' y pon tus PDFs dentro.")
        return None

    sys.stdout.write("--- Indexando normativa... ")
    sys.stdout.flush()

    loader = PyPDFDirectoryLoader("normativa/")
    docs = loader.load()

    splitter = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=100)
    chunks = splitter.split_documents(docs)

    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    vector_db = FAISS.from_documents(chunks, embeddings)
    sys.stdout.write("¡Listo! \n")

    retriever = vector_db.as_retriever(search_kwargs={"k": 5})

    tool = create_retriever_tool(
        retriever=retriever,
        name="buscador_normativa",
        description="Consulta para buscar información oficial sobre el ciclo, módulos y horas."
    )
    tools = [tool]

    llm = ChatGoogleGenerativeAI(
        model="gemini-2.5-flash-lite",  # Usamos la versión estable
        temperature=0,
        max_output_tokens=600,  # <-- (500-1000 es ideal para RAG)
        max_retries=2,
    )

    system_msg = (
        "Eres un asistente versátil y amable. Tu especialidad es ayudar con el Ciclo Formativo "
        "usando la herramienta 'buscador_normativa' para consultas específicas sobre módulos y
horas. "
        "Sin embargo, si el usuario te pregunta sobre otros temas generales (como cocina, cultura o
ayuda general), "
        "responde usando tu propio conocimiento de forma cordial."
    )

    prompt = ChatPromptTemplate.from_messages([
        ("system", system_msg),
        MessagesPlaceholder(variable_name="chat_history"),
        ("human", "{input}"),
        MessagesPlaceholder(variable_name="agent_scratchpad"),
    ])

    agent = create_tool_calling_agent(llm, tools, prompt)

    agent_executor = AgentExecutor(
        agent=agent,
        tools=tools,
        verbose=False,
        handle_parsing_errors=True
    )
```

```python
    # Memoria persistente durante la ejecución
    history = ChatMessageHistory()

    return RunnableWithMessageHistory(
        agent_executor,
        lambda session_id: history,
        input_messages_key="input",
        history_messages_key="chat_history",
    )

def limpiar_respuesta(salida_raw):
    """Extrae únicamente el texto de la respuesta de Gemini."""
    if isinstance(salida_raw, list):
        texto = ""
        for item in salida_raw:
            if isinstance(item, dict) and 'text' in item:
                texto += item['text']
            elif isinstance(item, str):
                texto += item
        return texto
    return str(salida_raw)

def chat_asistente():
    asistente = configurar_asistente()
    if not asistente: return

    print("\n" + "=" * 40)
    print("SISTEMA DE CONSULTA EDUCATIVA v2.5")
    print("   Escribe 'salir' para finalizar")
    print("=" * 40 + "\n")

    config = {"configurable": {"session_id": "sesion_docente"}}

    while True:
        usuario = input("Tú: ")
        if usuario.lower() in ["salir", "exit"]: break

        try:
            # Invocamos al agente
            response = asistente.invoke({"input": usuario}, config=config)

            # PASO CRÍTICO: Limpiamos la respuesta antes de mostrarla
            respuesta_final = limpiar_respuesta(response["output"])

            print(f"Asistente: {respuesta_final}\n")

        except Exception as e:
            print(f"Error en la comunicación: {e}")

if __name__ == "__main__":
    chat_asistente()
```

¡Llegamos al Final! Este script es la culminación de todo lo anterior. No es solo una cadena que sigue pasos; es un **Agente** que decide, tiene **memoria**

y sabe **limpiar** su propio rastro. Es, básicamente, lo más parecido a un asistente humano que podemos programar hoy en día.

Este script convierte nuestra IA en un ente capaz de razonar. A diferencia del ejercicio 3 (que siempre buscaba en el PDF), este agente **decide** si necesita el buscador o si puede responderte directamente basándose en la charla previa.

Explicación Detallada:

Componente	Función
AgentExecutor	El "director de orquesta". Gestiona el ciclo de pensamiento de la IA: *Pensar -> Actuar -> Observar -> Responder*.
Tool (Buscador)	Una capacidad extra. La IA no sabe qué hay en el PDF por defecto, así que le damos una "herramienta" (el retriever) y le explicamos en la description para qué sirve. Recalcar que la descripción de la herramienta es, en realidad, **Prompt Engineering**, si la descripción de la herramienta es mala, el agente no sabrá cuándo usarla. Es el 'manual de instrucciones' que lee la IA para saber qué herramientas tiene en su cinturón de utilidad".
ChatMessageHistory	El diario del asistente. Almacena las preguntas y respuestas anteriores para que la IA no olvide de qué estamos hablando.
limpiar_respuesta	**El filtro de seguridad.** Gemini 2.5 a veces envía datos técnicos (firmas, metadatos). Esta función nos da solo el texto, sin “basura”.

Vemos con más detalle desde el “prompt”:

1. El Prompt (ChatPromptTemplate)
Aquí definimos la estructura de la conversación. Es el "guion" que seguirá el agente.

- **prompt**: Tipo **ChatPromptTemplate**.

- **MessagesPlaceholder(variable_name="chat_history")**: Tipo **list[BaseMessage]**. Es un hueco dinámico donde LangChain inyectará automáticamente los mensajes anteriores de la base de datos de memoria.
- **"{input}"**: Tipo **str**. Es la pregunta actual del usuario.
- **MessagesPlaceholder(variable_name="agent_scratchpad")**: Tipo **list[BaseMessage]**.
    - **¿Qué es esto?** Es el "bloc de notas" del agente. Aquí se guardan los pensamientos intermedios de la IA y los resultados de las herramientas antes de darte la respuesta final. Es crucial para que el agente no entre en bucles infinitos.

---

2. El Agente (create_tool_calling_agent)
Es la lógica que decide: "¿Necesito usar una herramienta o respondo directamente?".

- **agent**: Tipo **Runnable**.
- **Funcionamiento**: Combina el modelo (llm), las herramientas (tools) y las instrucciones (prompt).
- **Importante**: Al usar tool_calling, le estamos diciendo a Gemini que, si ve que necesita datos del PDF, genere un código especial (un JSON) pidiendo permiso para ejecutar el buscador.

---

3. El Ejecutor (AgentExecutor)
Es el motor que realmente ejecuta el bucle de pensamiento, ciclo **ReAct** (Reason + Act), el AgentExecutor hace lo siguiente: **Piensa** (necesito el PDF), **Actúa** (llama a la tool), **Observa** (lee el resultado) y **Finaliza** (da la respuesta)".

- **agent_executor**: Tipo **AgentExecutor**.
- **handle_parsing_errors=True**: Esto es vital en entornos educativos. Si Gemini responde con un formato ligeramente "sucio", el ejecutor intentará limpiarlo en lugar de hacer que el programa explote.
- **verbose=False**: Si lo pones en True, verás en la consola todo el razonamiento interno de la IA ("Pensando...", "Llamando a herramienta...", "Observando...").

---

4. La Memoria (ChatMessageHistory)
Donde se guardan físicamente los mensajes. Aclarar que ChatMessageHistory() se borra al cerrar el script. En el mundo real, conectaríamos esto a una base de datos como Redis o PostgreSQL para que el bot recordase al usuario al día siguiente.

- **history**: Tipo **ChatMessageHistory**.

- **Contenido**: Es esencialmente una lista de objetos que crecen con cada interacción: [HumanMessage, AIMessage, HumanMessage, AIMessage...]. Vive en la memoria RAM del programa mientras la sesión está activa.

---

5. El Envoltorio Maestro (RunnableWithMessageHistory)
Esta es la "capa mágica" que hace que no tengamos que gestionar la memoria a mano.

- **Tipo de dato**: **RunnableWithMessageHistory**.
- **input_messages_key="input"**: Le dice a la memoria: "Busca la pregunta del usuario en la clave 'input'".
- **history_messages_key="chat_history"**: Le dice a la memoria: "Inyecta todo el pasado en el hueco 'chat_history' del prompt".

Resumen:

Variable	Tipo de Dato	Función Real
prompt	ChatPromptTemplate	La **plantilla** de la conversación.
agent	Runnable	El **razonamiento** (el algoritmo de decisión).
agent_executor	AgentExecutor	El **motor** que corre el agente y llama a las tools.
history	ChatMessageHistory	El **almacén** de mensajes (la base de datos).
Runnable...	Runnable	El **automatizador** que pega la memoria al agente.

Ejercicio Propuesto: "Implementación de un Agente Autónomo con Doble Capacidad de Consulta"

Objetivo: Desarrollar la lógica de decisión de un agente para que sea capaz de discriminar entre diferentes fuentes de información especializadas.

Instrucciones para el alumno:

1. **Creación de la Segunda Tool:** Define una función en Python (por ejemplo, consultar_calendario_examenes()) que devuelva un diccionario o cadena con fechas clave (ej: "Proyecto Web: 15 de junio").
2. **Registro de Herramientas:** Configura el agente para que tenga acceso tanto a la herramienta de búsqueda en PDF (buscador_normativa) como a la nueva herramienta de calendario.
3. **Refinamiento del Razonamiento:** Ajusta la descripción de las herramientas para que la IA sepa cuándo acudir a cada una.
4. **Validación de Memoria:** Realiza una consulta que requiera recordar el contexto anterior para usar la segunda herramienta (ej: Preguntar por un módulo y luego decir "¿Y cuándo es el examen?").

Sugerencia: Usar el decorador @tool de LangChain, que es la forma más "limpia" y moderna de crear herramientas nuevas en Python.

El Decálogo del Ingeniero de IA

Sigue estas reglas de buenas prácticas:

1. **La Regla del "Basura entra, Basura sale" (GIGO):** Si tu PDF está mal escaneado o el texto está desordenado, ningún modelo del mundo (ni Gemini 3 Ultra) podrá responder bien. **Limpia tus datos antes de indexar.**
2. **No uses un Agente si basta con una Cadena:** Los agentes son lentos y caros porque "piensan" antes de actuar. Si solo quieres resumir un texto, usa una Chain simple. Usa Agents solo cuando necesites que la IA decida qué herramienta usar.
3. **El Prompt es el Código:** Trata tus system_msg como código fuente. Un cambio en una palabra ("responde corto" vs "sé conciso") puede cambiar totalmente el consumo de tokens y la precisión.
4. **Vigila la Ventana de Contexto:** No intentes meter 50 fragmentos del PDF de golpe. La IA se aturde y empieza a ignorar información (efecto "Lost in the Middle"). Quédate con el **Top-K (3 a 5).**

5. **Cita tus Fuentes:** Un RAG profesional siempre dice: "Según la página 4...". Si no hay fuente, el usuario no confiará en el bot. Usa los **metadatos** de LangChain.
6. **Temperatura 0 es tu mejor amiga:** Para aplicaciones técnicas, de normativa o educativas, la creatividad es el enemigo. Mantén la `temperature=0` para evitar inventos.
7. **Cuidado con los Bucles Infinitos:** Los agentes a veces se quedan pensando: *"Uso la herramienta... no me sirve... vuelvo a usarla"*. Configura siempre un `max_iterations` en tu `AgentExecutor`.
8. **Seguridad y Privacidad:** Nunca, jamás, subas el archivo `.env` a GitHub. Usa variables de entorno del sistema en producción.
9. **La IA es un Asistente, no un Oráculo:** Diseña la interfaz para que el usuario sepa que la respuesta puede contener errores. Incluye siempre un botón de *"Reportar error"*.
10. **Evalúa con Datos, no con Sensaciones:** No digas "parece que funciona bien". Pásale 10 preguntas fijas y mide cuántas veces acierta. Eso es **AI Engineering**.

Siguientes Pasos

1. **Despliegue:** ¿Cómo subir esto a una API con FastAPI?
2. **Interfaz:** ¿Cómo conectar este script de Python a un frontend en React o Streamlit?